WHY

코드를 엔지니어링 지식으로 읽는 법

파일을 요약하지 않고, 성립 조건과 실패 압력에서 설계를 재구성한다

> **중요한 것은 “무슨 코드가 있는가”가 아니라 “어떤 약속을 지키려면 이 코드가 필요해지는가”다.**

대상 snapshot
v2.1.88 / 2ca5ddabfed5

설명 구조
조건 · 이론 · 구현 · 반례

제3자 복원 소스에 대한 교육용 분석이다. 코드 관찰, 이론적 비유, 개선 제안을 구분하며 누락 모듈을 추정해 채우지 않는다.

분석 대상과 지식 대상을 분리한다

> 복원 저장소에서 무엇까지 말할 수 있는가?

성립 조건

코드는 실제로 존재하지만 공식 원본·완전한 빌드·장기 이력이 아니다.

이론 렌즈

W3C PROV의 관점에서 source entity, 분석 activity, 보고서 entity를 서로 다른 provenance 대상으로 둔다.

코드의 구체화

고정 commit의 파일·행은 관찰 사실로, 설계 이유는 inference로, 개선 방향은 proposal로 표기한다.

깨지는 지점

코드가 존재한다는 이유만으로 upstream 의도나 production 활성 상태를 단정하면 reconstruction bias가 된다.

해석 이 보고서는 원전을 복원하지 않는다. 관찰 가능한 구현이 닫으려는 문제와 그 한계를 재구성한다.

근거와 연결 README_KR.md · repository bias control · W3C PROV-O

왜 채팅 앱이라는 모델이 부족한가

> 사용자가 보는 화면과 시스템이 책임지는 결과 사이에는 무엇이 있는가?

성립 조건

코딩 에이전트는 텍스트를 생성하는 데서 끝나지 않고 파일·프로세스·세션을 변화시킨다.

이론 렌즈

언어 처리계는 description을 실행 가능한 표현으로 낮추고, 보호계는 subject의 요청을 resource effect 앞에서 중재한다.

코드의 구체화

Tool 계약, schema parse, permission, tool-specific validation, call, result pairing이 모델 출력과 저장소 효과 사이에 놓인다.

깨지는 지점

대화 메시지만 추적하면 stale write, denied effect, partial execution, resume 불일치를 설명하지 못한다.

해석	가장 생산적인 모델은 “대화 UI”가 아니라 “모델이 제안한 효과를 통제하여 해석하는 런타임”이다.

근거와 연결 src/Tool.ts · src/services/tools/toolExecution.ts · NIST SP 800-207

학습 질문이 자료의 범위를 결정한다

> 어떤 질문에 답하지 못하면 이 프로젝트를 이해했다고 할 수 없는가?

질문	피상적 답	요구되는 설명
context	prompt를 합친다	선택·순서·provenance·cache·lossy compact의 결합
tool	함수를 호출한다	표현 유효성·권한·현재 상태·효과·관찰의 분리
turn	generator로 반복한다	부분 효과 뒤 protocol과 durable state의 합류
UI	React로 그린다	canonical state와 interactive projection의 경쟁
refactor	큰 파일을 나눈다	불변조건 owner와 동등성 oracle을 보존하는 순서

이 질문들은 파일 구조가 아니라 제품 약속에서 출발한다. 코드 탐색은 답을 찾는 수단이며, 질문 자체가 폴더 목록으로 변하지 않는다.

근거와 연결 progressive knowledge acquisition · knowledge demand

한 페이지는 하나의 인과 주장을 완성한다

왜 순서도만으로는 설계를 설명할 수 없는가?

성립 조건 × 이론 렌즈 × 코드 구체화 × 실패 반례 → 설계 의미

실행 순서는 “무엇 다음에 무엇”을 보여 주지만, 왜 경계가 필요한지와 어떤 조건에서 비유가 깨지는지는 보여 주지 않는다. 따라서 본문은 원인과 제약을 먼저 설명하고, 순서는 race·pairing·settlement처럼 시간 자체가 불변조건인 경우에만 사용한다.

독해 규칙	코드 이름을 이론 용어로 치환하지 않는다. 유사성을 만드는 관찰과 유사성이 끝나는 한계를 함께 적는다.

사실·추론·비유·제안을 같은 문장에 섞지 않는다

> “이 코드는 transaction이다”라는 말은 어느 수준에서 참인가?

종류	허용되는 말	필요한 증거
관찰 사실	특정 branch가 tombstone을 삽입한다	고정 revision 코드 좌표
source-backed inference	pairing invariant를 보존하려는 구조다	여러 코드 관찰 + 반례
bounded analogy	turn은 saga와 닮았다	공통 구조 + 차이와 종료점
proposal	journal과 projection을 분리할 수 있다	목표 scenario + 위험 + oracle

비보상 원칙 코드 좌표가 많아도 잘못된 개념 연결을 보상하지 못하고, 유명한 이론도 직접 코드 관찰을 대체하지 못한다.

근거와 연결 decision rubric knowledge

서로 다른 추상화 수준을 연결하되 합치지 않는다

> 언어 기능, 패턴, 제품 책임은 어떤 관계인가?

01	**언어 메커니즘**	async generator, discriminated union, AbortSignal, dynamic import
02	**구현 패턴**	stream protocol, optimistic validation, projection, registry
03	**capability**	context compilation, authorized effect, resume, provider adaptation
04	**제품 불변조건**	잘못된 파일을 덮어쓰지 않음, 효과와 관찰의 pairing, 재개 정합성
05	**운영 가치**	안전·신뢰성·latency·호환성·변경 가능성

아래 층은 위 층을 실현하지만 동일하지 않다. generator를 제거해도 protocol을 다른 방식으로 보존할 수 있고, generator를 사용해도 cancellation과 durability는 자동으로 생기지 않는다.

이론은 이름표가 아니라 반례 생성기다

> 배경지식은 코드 설명을 어떻게 더 엄격하게 만드는가?

이론	설명하는 압력	검사하게 만드는 반례
compiler IR	heterogeneous context의 lowering	변환 뒤 provenance·순서·타입 손실
reference monitor	effect 앞의 지속적 중재	schema 통과를 권한으로 오인
OCC / TOCTOU	읽은 상태와 쓸 상태의 시간차	검사와 write 사이 await
Saga	되돌릴 수 없는 부분 효과	retry가 같은 명령을 중복 실행
projection	여러 소비자 표현	UI state를 durable truth로 승격
product line	build/provider별 가변성	gate 밖 static import로 제외 코드 유입

저장소 지식을 먼저 쓰고 외부 이론은 빈틈만 채운다

왜 외부 자료가 많을수록 좋은 보고서가 아닌가?

성립 조건

현재 저장소는 이미 지식 lifecycle, projection, bias, tradeoff, operating model을 보유한다.

이론 렌즈

Progressive acquisition은 competency question과 gap을 먼저 만들고, 필요한 증거만 추가한다.

코드의 구체화

local owner 문서가 설명 방법과 경계를 제공하고, 외부 specification·원논문은 언어/보안/동시성의 residual gap을 채운다.

깨지는 지점

검색 결과를 chapter 목록으로 바꾸면 repository context와 source applicability가 사라진다.

해석 외부 출처의 역할은 권위를 빌리는 것이 아니라 현재 해석을 반증하거나 경계를 정교하게 하는 것이다.

근거와 연결 artifacts/knowledge/README.md · docs/reference/README.md · artifacts/knowledge/operations/progressive-knowledge-acquisition-and-modeling.md

코드 분포는 중요도가 아니라 탐색 신호다

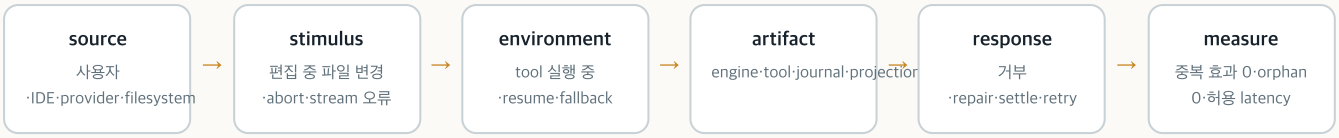
> 39,050줄의 shell 관련 코드가 무엇을 증명하는가?

분량은 구현 비용과 edge-case 표면을 보여 주지만 제품 중요도를 직접 측정하지 않는다. comment의 security marker 역시 실제 취약점 수가 아니다. 분포는 “어디에서 어떤 설명 질문을 더 물을 것인가”를 정하는 탐색 prior로만 쓴다.

관찰	말할 수 있음	말할 수 없음
shell/security 밀도	parser·platform·policy pressure가 넓다	Bash가 가장 중요한 기능이다
generator 55개	메커니즘이 희소하다	모든 generator가 핵심 turn에 속한다
logEvent 1,088회	관측 seam이 넓다	운영 품질이 우수하다
큰 REPL 파일	책임 후보가 집적됐다	분해하면 자동으로 cohesion이 좋아진다

“좋은 구조”를 stimulus와 response로 바꾼다

> 리팩터링 논의를 어떻게 검증 가능한 질문으로 만드는가?



이 구조는 “maintainability가 좋아진다”를 작은 검증 문제로 내린다. 실제 response measure는 upstream owner가 없으므로 보고서가 임의로 확정하지 않는다.

근거와 연결 artifacts/knowledge/architecture/architecture-characteristics-and-tradeoffs.md · SEI QAW

다섯 책임이 하나의 코드 변경을 성립시킨다

> 프로젝트 핵심을 가장 작게 표현하면 무엇인가?

Evidence-bearing Context → Proposed Program → Authorized Effect → Typed Observation → Durable Reconciliation

Context는 단순 token 묶음이 아니라 출처와 순서를 가진 실행 입력이다. Proposed Program은 tool name과 arguments로 표현된다. Effect는 permission과 현재 상태를 통과해야 한다. Observation은 같은 tool id와 결합해야 하며, reconciliation은 다음 turn·transcript·resume가 같은 과거를 보게 한다.

핵심성 판정	이 다섯 책임을 직접 소유하거나 그 불변조건을 지지하지 않는 기능은 확장·adapter·projection으로 분류한다.

영상의 통찰을 중심축 안에 배치한다

> generator와 UI를 어떻게 과대평가하지 않으면서 활용하는가?

영상 주제	유효한 통찰	보고서에서의 위치
async generator	suspension과 progressive observation	turn protocol의 시간 표현
React/Ink UI	terminal을 state projection으로 구성	interactive adapter와 canonical state 경계

두 주제는 큰 축의 일부다. 영상의 설명은 코드를 읽는 hypothesis로 쓰고, ECMAScript·React/Ink owner 자료와 실제 구현으로 범위를 교정한다.

학습은 설명을 재현하는 것이 아니라 반례를 만드는 일이다

> 한 개념을 이해했는지 어떻게 확인하는가?

01	**명제**	“Edit는 stale write를 막는다”처럼 falsifiable하게 쓴다.
02	**전제**	read state가 있고 같은 대상 identity를 비교한다.
03	**반례**	검사 뒤 write 전에 외부 변경을 삽입한다.
04	**관찰**	효과 발생 여부·오류·transcript·resume state를 함께 본다.
05	**구조 결론**	어떤 owner와 critical section이 불변조건을 가져야 하는지 설명한다.

여섯 권은 기능이 아니라 문제 경계로 나뉜다

어떤 순서로 읽어야 인과 관계가 유지되는가?

권	문제 경계	중심 질문
01	context compiler	모델이 보게 되는 세계는 어떻게 만들어지는가
02	effect interpreter	제안은 언제 실제 행동이 될 자격을 얻는가
03	turn semantics	부분 효과 뒤 한 turn을 어떻게 닫는가
04	projection	UI·SDK·transcript는 어떤 진실을 보여 주는가
05	evolution	불변조건을 잃지 않고 구조를 어떻게 바꾸는가

읽기 경로 처음에는 00→01→02→03을 읽고, UI 관심자는 04로, 리팩터링·운영 관심자는 05로 이어 간다.

상세함은 확실성의 과장이 아니다

> 무엇을 끝까지 unknown으로 남기는가?

누락된 `src/types/message.ts`와 `src/query/transitions.ts`, incomplete build, shallow history, 실제 provider cohort와 production metric, upstream roadmap은 복원할 수 없다. 이 자료는 그 빈칸을 architecture pattern으로 메우지 않는다.

> **모르는 것을 지우지 않는 것이 상세한 설명의 일부다.**

근거와 연결 README_KR.md · knowledge lifecycle rubric operation